APPLICATION DEPLOYMENT (STUDENT-MANAGEMENT SYSTEM)

CREATE A UBUNTU SERVER

1. apt update
2. apt install openjdk-21-jdk -y
3. apt install maven -y
4. java --version
5. mvn --version

CREATE A MYSQL DATABASE

1. user = admin
2. password = admin123
3. allow sg (mysql-3306)

CONFIGURE DATABASE

1. apt install mysql-client -y
2. mysql -u admin -h <db_endpoint> -padmin123
    1. create database school_db;
    2. CREATE TABLE IF NOT EXISTS students (
       id INT AUTO_INCREMENT PRIMARY KEY,
       name VARCHAR(100) NOT NULL,
       email VARCHAR(100) NOT NULL,
       course VARCHAR(100) NOT NULL,
       marks DECIMAL(5,2) NOT NULL DEFAULT 0
       );

CONFIGURE APPLICATION ON UBUNTU

1. git clone https://github.com/mayurpandit25/student-management.git
2. cd student-management
3. cd src/main/resources
4. vim db.properties
   db.url=jdbc:mysql://<RDS-ENDPOINT>:3306/school_db?useSSL=false&serverTimezone=UTC
   db.user=admin
   db.password=admin123
5. cd student-management
6. mvn clean package
7. ls target/

INSTALLING TOMCAT

1. wget https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.57/bin/apache-tomcat-10.1.57.tar.gz
2. tar -xzf apache-tomcat-10.1.57.tar.gz

DEPLOY APPLICATION ON TOMCAT

1. cp -rf student-management/target/student-management.war apache-tomcat-10.1.57/webapps/student.war
2. cd apache-tomcat-10.1.57/

3. cd bin
4. ./catalina.sh start

TEST APPLICATION INTERNALLY

1. curl http://localhost:8080/student

TEST APPLICATION EXTERNALLY

2. http://public_ip:8080/student

MAP EC2 INSTANCE TO DOMAIN (HOSTINGER)

1. Go to Hostinger
2. click on domain name (mayurcbz.space)
3. click on DNS nameservers
4. Go to Manage DNS records
5. Edit already created A record
    1. value = (your ec2 instance public_ip)
    2. TTL = 60
    3. update

CONFIGURE REVERSE PROXY ON NGINX (To hide backend servers)

1. apt update
2. apt install nginx -y
3. systemctl start nginx
4. vim /etc/nginx/sites-available/student

```
server {
   listen 80;
   server_name mayurcbz.space www.mayurcbz.space;

   location / {
      proxy_pass http://localhost:8080;

      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
      proxy_set_header X-Forwarded-Proto $scheme;
   }
}
```

5. nginx -t
6. ln -s /etc/nginx/sites-available/student /etc/nginx/sites-enabled/
7. rm -rf /etc/nginx/sites-enabled/default
8. systemctl restart nginx

TEST THE APPLICATION

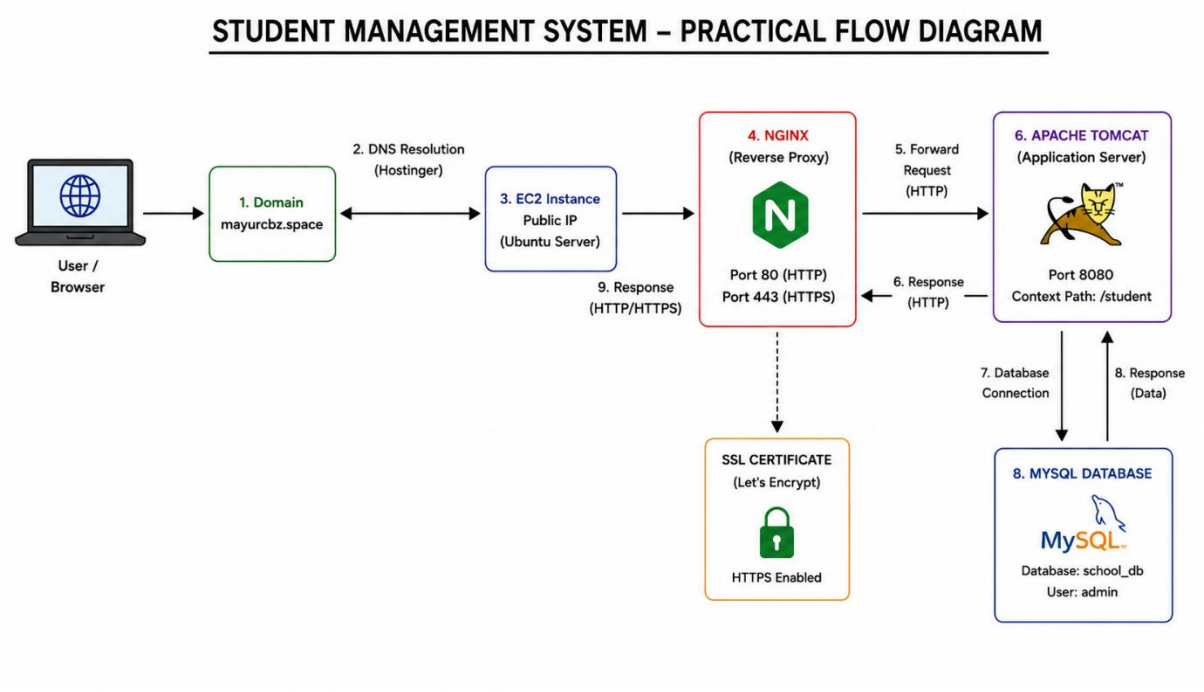
1. http://public_ip:8080/student -----> Before Reverse proxy
2. http://public_ip/student ----------> After Reverse proxy
3. http://mayurcbz.space/student ---> Using Domain

CONFIGURE HTTPS (SSL) USING NGINX

1. apt install certbot python3-certbot-nginx -y
2. certbot --version
3. certbot --nginx
    1. Email Address = (mayur@gmail.com)
    2. Accept Terms = (Y)
    3. Redirect HTTP to HTTPS = (Y)
4. systemctl restart nginx

TEST THE APPLICATION USING HTTPS

1. https://mayurcbz.space/student



EXPLANATION

NGINX

Nginx is a Web server that host the static webpages and also provides the features like

1. Reverse proxy
2. Load balancing
3. SSL terminator

Without nginx

Users request is going directly to the backend server

Browser ----------> Tomcat (8080)

With Nginx
Users request handle by the nginx then redirect to the backend servers.
Browser ---------> Nginx (80/443) ----------> Tomcat (8080)

1.Reverse Proxy

1. A Reverse Proxy is a server that receives client requests and forwards them to one or more backend servers.
2. So, the client never communicates directly with the backend.

Without Reverse Proxy
Browser ----------> Tomcat (8080)
Users know
1. backend IP
2. backend port

With Reverse Proxy
Browser ----------> Nginx --------> Tomcat (8080)
1. The browser only knows Nginx.
2. Tomcat remain the hidden.

```
server {
   listen 80;
   server_name _;
   location / {
      proxy_pass http://localhost:8080;
   }
}
```

How Reverse Proxy Works
User opens
https://mayurcbz.space/student

Nginx receives the request.

Nginx forwards it to
http://localhost:8080/student

Tomcat processes it.
Tomcat sends the response back to Nginx.
Nginx returns the response to the browser.

---

3. Load Balancing

Load Balancing means distributing incoming requests across multiple backend servers so that no single server becomes overloaded.

CONCEPT OF LOAD BALANCER

Suppose 3 Tomcat servers exist.

```
          Nginx
        /   |   \
 Tomcat1  Tomcat2  Tomcat3
```

1000 users visit the website.
Instead of sending all requests to one Tomcat server, Nginx distributes them.

IT SIMPLY USES THE LOAD BALANCER ALGORITHEMS TO SEND THE TRAFFIC

1. Round Robin
2. Weighted Round Robin
3. IP hash
4. least connections
5. least response time

```
upstream backend {
   server 10.0.1.10:8080;
   server 10.0.1.11:8080;
   server 10.0.1.12:8080;
}

server {
   listen 80;
   location / {
      proxy_pass http://backend;
   }
}
```

---

3. SSL (HTTPS)

SSL (Secure Sockets Layer) encrypts communication between the browser and the server. Although modern systems use TLS, the term SSL is still commonly used.

Without SSL

1. Browser -------> HTTP Request -----> Internet -------> Server
2. Data travels in plain text.
3. Passwords and sensitive information can potentially be intercepted.

With SSL

1. Browser -------> Encrypted HTTPs Request -----> Internet -------> Server
2. The data is encrypted before leaving the browser and decrypted by the server.

Why We Use Certbot for Creating Certificates

Certbot:

1. Requests a free SSL certificate from Let's Encrypt
2. Installs the certificate
3. Configures Nginx automatically
4. Renews the certificate before it expires

Why Nginx is Used

1. Acts as a Web Server
2. Works as a Reverse Proxy
3. Performs Load Balancing
4. Handles SSL/TLS
5. Serves static files efficiently
6. Hides backend servers
7. Improves security and performance

APACHE TOMCAT

Apache Tomcat is a Java Application Server.

Why do we need Tomcat?

1. After building the application using Maven,
2. a WAR file is created. (like student.war)
3. A browser cannot execute a WAR file directly.
4. Tomcat understands Java Servlets and JSPs, deploys the WAR file, and makes the application accessible through HTTP.
5. It automatically
    1. detects the war file
    2. Extracts it
    3. Deploys the application
    4. makes it available

Why does Tomcat use Port 8080

By default:

- **80** → HTTP (Web Servers like Nginx & Apache)
- **443** → HTTPS (SSL)
- **8080** → Java Application Server (Tomcat)

Keeping Tomcat on 8080 allows Nginx to handle HTTP/HTTPS traffic on the standard web ports.

Apache Tomcat Directory Structure

apache-tomcat-10.1.57/
├— bin/
├— conf/
├— lib/
├— logs/
├— temp/
├— webapps/
├— work/
└— webapps/

1.bin
The bin directory contains startup and shutdown scripts used to control the Tomcat server.

2.conf
The conf folder stores all Tomcat configuration files.

3.lib
The lib folder contains shared Java libraries (JAR files)

4.webapps
Tomcat deploys applications from the webapps directory.

5.logs
Stores Tomcat log files.

6.temp
Stores temporary files while Tomcat is running.

7.work
Stores compiled JSP files and cached data.
